

Situation d'Apprentissage et d'Évaluation (SAÉ)
S1.04
**Création d'une base de données : Rapportage de la
station balnéaire**

Rapport réalisé par : PATHMARANJAN Kuruparan
Département : Informatique
Groupe : Venus
Enseignant(e) responsable du groupe : Mme ZARG AYOUNA
Responsable de matière : M. AUDIBERT

Partie 1 : Script manuel de création de la base de données

```
CREATE TABLE regions (  
    code INTEGER PRIMARY KEY,  
    nom VARCHAR NOT NULL);
```

```
CREATE TABLE departements (  
    code INTEGER PRIMARY KEY,  
    region INTEGER REFERENCES regions ON DELETE CASCADE,  
    nom VARCHAR NOT NULL);
```

```
CREATE TABLE communes (  
    code INTEGER PRIMARY KEY,  
    departement INTEGER REFERENCES departements ON DELETE CASCADE,  
    nom VARCHAR NOT NULL);
```

```
CREATE TABLE sites (  
    idSite INTEGER PRIMARY KEY,  
    nom VARCHAR NOT NULL,  
    codeCommune INTEGER REFERENCES communes ON DELETE CASCADE,  
    dateDeclaration DATE NOT NULL,  
    typeEau VARCHAR NOT NULL,  
    longitude FLOAT NOT NULL,  
    latitude FLOAT NOT NULL);
```

```
CREATE TABLE evenements (  
    idEvenement INTEGER PRIMARY KEY,  
    idSite INTEGER REFERENCES sites ON DELETE CASCADE,  
    evenement VARCHAR NOT NULL,  
    debut DATE NOT NULL,  
    fin DATE NOT NULL,  
    mesure VARCHAR NOT NULL);
```

```
CREATE TABLE analyses (  
    idAnalyse INTEGER PRIMARY KEY,  
    idSite INTEGER REFERENCES sites ON DELETE CASCADE,  
    datePrelevement DATE NOT NULL,  
    enterocoques INTEGER NOT NULL,  
    escherichia INTEGER NOT NULL);
```

Partie 2 : Modélisation et script de création "avec AGL"

Atelier de Génie Logiciel (AGL) choisit : **pgModeler**

Association fonctionnelle :

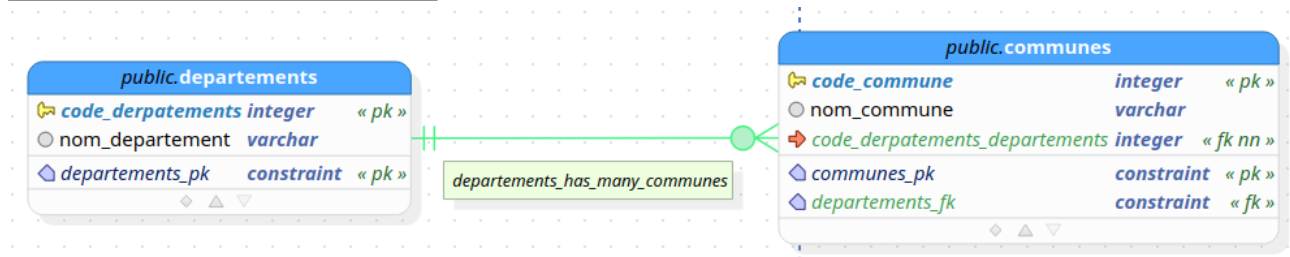


Figure 1 : AGL association fonctionnelle

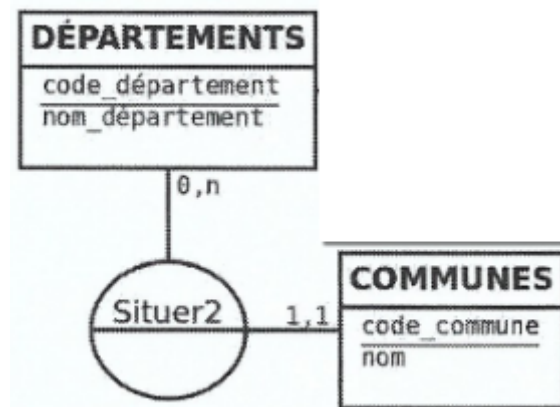


Figure 2 : Modèle type association fonctionnelle

La lecture d'un modèle de conception sur AGL comme sur pgModeler est différent du modèle de conception fait en cours.

Dans la table **departements**, il y a une colonne **code_departements** qui reçoit des entiers (**integer**) et est une clé primaire (primary key) que l'on peut voir par "pk". Ensuite, il y a une autre colonne qui correspond au **nom_department** qui prend des caractères variables.

Par la suite, il y a une deuxième table **colonnes** qui contient lui aussi deux colonnes ; **code_commune** qui peut contenir des entiers (**integer**) et **nom_commune** qui peut contenir tout type de caractères (**varchar**).

On voit qu'il y a un lien entre ces deux tables : **une association fonctionnelle** car une commune peut être affectée à un seul département. Ainsi, lors du lien, le logiciel illustre avec un trait qui commence de la table **departements** et va vers **communes** mais se divise en plusieurs chemin avant la table **communes** qui montre qu'il peut y avoir plusieurs communes. On remarque que la colonne **code_departements** est automatiquement ajoutée à la table **communes**. La lecture ici est simple car il est écrit : "departemnts_has_many_communes" ainsi, on comprend que chaque département peut avoir plusieurs communes mais pas inversement. Pour montrer l'utilisation des attributs de la table **departements** dans la table **communes**, il est marqué "fk" pour **foreign key** (clé étrangère).

La modélisation faite en cours (modèle conceptuel) montre deux tables contenant les mêmes attributs que celui de l'AGL avec un lien entre les deux faites par une bulle centrale nommée 'Situer2' et les cardinalités correspondant. Les cardinalités (**0,n** côté département et **1,1** côté commune) indiquent qu'un département peut regrouper plusieurs communes, alors qu'une commune appartient à un seul département.

Association maillée :

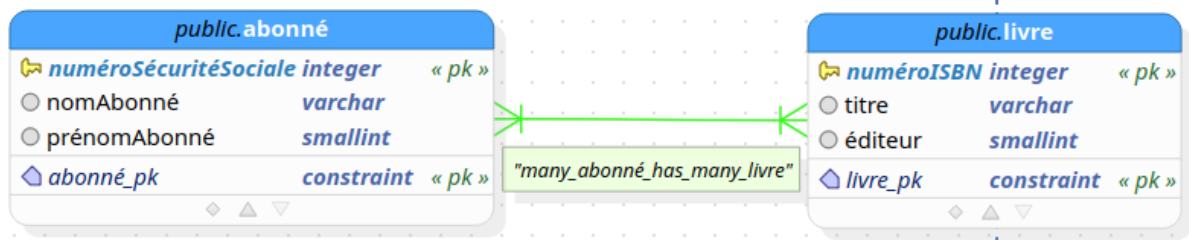


Figure 3: AGL association maillée

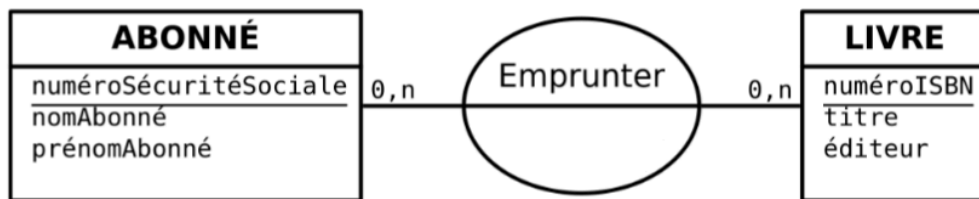
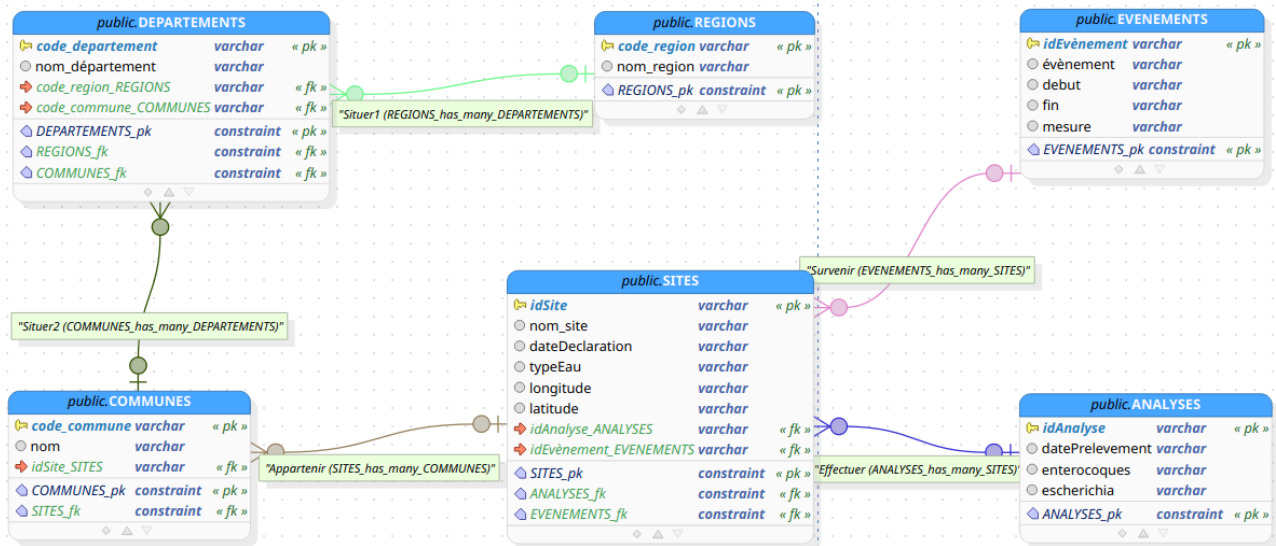


Figure 4 : Modèle type association fonctionnelle

Sur la table **abonné**, il y a trois colonnes : **numéroSécuritéSociale** (integer / primary key), **nomAbonné** (varchar) et **prénomAbonné** (varchar). Et dans la table **livre**, il y a trois colonnes : **numéroISBN** (integer / primary key), **titre** (varchar) et **éditeur** (varchar). Sur la modélisation faite sur AGL, il y a un trait qui montre l'association et pour comprendre quelle est l'association, on doit voir la façon dont se divisent les traits, ici ils commencent divisé et se terminent divisé. Il est aussi écrit "many_abonné_has_many_livre" qui signifie "plusieurs à plusieurs" ainsi chaque abonné peut avoir plusieurs livres associés.

Alors que dans le modèle vu en cours, il y a deux tables reliées par une bulle nommée "Emprunter". On voit que c'est une association maillée car la cardinalité des deux côtés est **0,n**.

Modèle physique de données correspondant au modèle conceptuel de données représenté figure 1 (sujet Saé)



Script SQL de création des tables généré automatiquement par l'AGL à partir du modèle précédent

```
CREATE DATABASE new_database;
```

```
CREATE TABLE public."REGIONS" (  
  code_region varchar NOT NULL,  
  nom_region varchar,  
  CONSTRAINT "REGIONS_pk" PRIMARY KEY (code_region)  
);
```

```
CREATE TABLE public."DEPARTEMENTS" (  
  code_departement varchar NOT NULL,  
  "nom_departement" varchar,  
  "code_region_REGIONS" varchar,  
  "code_commune_COMMUNES" varchar,  
  CONSTRAINT "DEPARTEMENTS_pk" PRIMARY KEY (code_departement)  
);
```

```
ALTER TABLE public."DEPARTEMENTS" ADD CONSTRAINT "REGIONS_fk" FOREIGN  
KEY ("code_region_REGIONS")  
REFERENCES public."REGIONS" (code_region) MATCH FULL  
ON DELETE SET NULL ON UPDATE CASCADE;
```

```
CREATE TABLE public."COMMUNES" (  
  code_commune varchar NOT NULL,  
  nom varchar,  
  "idSite_SITES" varchar,  
  CONSTRAINT "COMMUNES_pk" PRIMARY KEY (code_commune)  
);
```

```
ALTER TABLE public."DEPARTEMENTS" ADD CONSTRAINT "COMMUNES_fk"  
FOREIGN KEY ("code_commune_COMMUNES")  
REFERENCES public."COMMUNES" (code_commune) MATCH FULL  
ON DELETE SET NULL ON UPDATE CASCADE;
```

```
CREATE TABLE public."SITES" (  
  "idSite" varchar NOT NULL,  
  nom_site varchar,  
  "dateDeclaration" varchar,  
  "typeEau" varchar,  
  longitude varchar,  
  latitude varchar,  
  "idAnalyse_ANALYSES" varchar,  
  "idEvènement_EVENEMENTS" varchar,  
  CONSTRAINT "SITES_pk" PRIMARY KEY ("idSite")  
);
```

```
ALTER TABLE public."COMMUNES" ADD CONSTRAINT "SITES_fk" FOREIGN KEY  
("idSite_SITES")  
REFERENCES public."SITES" ("idSite") MATCH FULL  
ON DELETE SET NULL ON UPDATE CASCADE;
```

```
CREATE TABLE public."ANALYSES" (
  "idAnalyse" varchar NOT NULL,
  "datePrelevement" varchar,
  enterocoques varchar,
  escherichia varchar,
  CONSTRAINT "ANALYSES_pk" PRIMARY KEY ("idAnalyse")
);
```

```
CREATE TABLE public."EVENEMENTS" (
  "idEvènement" varchar NOT NULL,
  "évènement" varchar,
  debut varchar,
  fin varchar,
  mesure varchar,
  CONSTRAINT "EVENEMENTS_pk" PRIMARY KEY ("idEvènement")
);
```

```
ALTER TABLE public."SITES" ADD CONSTRAINT "ANALYSES_fk" FOREIGN KEY
("idAnalyse_ANALYSES")
REFERENCES public."ANALYSES" ("idAnalyse") MATCH FULL
ON DELETE SET NULL ON UPDATE CASCADE;
```

```
ALTER TABLE public."SITES" ADD CONSTRAINT "EVENEMENTS_fk" FOREIGN KEY
("idEvènement_EVENEMENTS")
REFERENCES public."EVENEMENTS" ("idEvènement") MATCH FULL
ON DELETE SET NULL ON UPDATE CASCADE;
```

Commentaire sur les différences entre les scripts produits manuellement et automatiquement

Un script produit manuellement est écrit par un humain donc peut contenir des erreurs, être sans détails (commentaires) mais peut-être corrigé. Alors qu'un script automatique est dans la plupart des cas généré par l'AGL donc est vraiment structuré, contient toutes les informations, est un script parfait généré des tables fait par l'humain. Cependant, lors de l'exécution du script généré automatiquement par l'AGL à partir du modèle précédent (figure 1 sujet Saé), il y a eu une erreur sur cette commande :

```
ALTER TABLE public."DEPARTEMENTS" OWNER TO postgres;
```

qui renvoie l'erreur :

```
ERROR: must be able to SET ROLE "postgres"
```

Cela signifie que l'utilisateur (moi-même) ne possède pas la permission pour exécuter cette commande et qu'il faut être "super utilisateur" pour le faire.

Partie 3 : Peuplement des tables

Script de peuplement de la base de données

```
CREATE TEMP TABLE depart_fr (code VARCHAR, nom VARCHAR, code_region
VARCHAR, nom_region VARCHAR);
```

```
\copy "depart_fr" (code, nom, code_region, nom_region) FROM
'~/Desktop/SAE_BD/departements-france.csv' DELIMITER ',' CSV HEADER;
```

```
CREATE TEMP TABLE sit (saison VARCHAR, region VARCHAR, depart
VARCHAR, code VARCHAR, preced_code VARCHAR, evolution VARCHAR, nom
VARCHAR, code_INSEE VARCHAR, commune VARCHAR, date_declar
VARCHAR, type_eau VARCHAR , long VARCHAR, lat VARCHAR);
```

```
\copy sit FROM
~/Desktop/SAE_BD/liste-des-sites-de-baignade-saison-balneaire-2024.csv
DELIMITER ';' CSV HEADER ENCODING 'LATIN1';
```

```
CREATE TEMP TABLE res (saison VARCHAR ,region VARCHAR ,depart
VARCHAR ,code VARCHAR ,date VARCHAR ,res_enter VARCHAR ,res_esch
VARCHAR ,statut VARCHAR,other1 VARCHAR, other2 VARCHAR, other3
VARCHAR );
```

```
\copy res FROM
~/Desktop/SAE_BD/saison-balneaire-2024-resultats-danalyses.csv DELIMITER ';'
CSV HEADER ENCODING 'LATIN1';
```

```
CREATE TEMP TABLE info (saison VARCHAR, region VARCHAR, depart
VARCHAR, code VARCHAR, type_even VARCHAR, debut VARCHAR, fin
VARCHAR, mesure VARCHAR);
```

```
\copy info FROM
~/Desktop/SAE_BD/saison-balneaire-2024-informations-sur-la-saison.csv
DELIMITER ';' CSV HEADER ENCODING 'LATIN1';
```

```
INSERT INTO "REGIONS" (code_region , nom_region) SELECT DISTINCT
code_region,nom_region FROM depart_fr ;
```

```
INSERT INTO "SITES" ("idSite", "nom_site", "dateDeclaration", "typeEau",
"longitude", "latitude") SELECT DISTINCT code, nom, date_declar, type_eau, long,
lat FROM sit ;
```

```
ALTER TABLE "ANALYSES" DROP CONSTRAINT "ANALYSES_pk" CASCADE ;
```

```
INSERT INTO "ANALYSES"
("idAnalyse","datePrelevement","enterocoques","escherichia") SELECT DISTINCT
code,date,res_enter,res_esch FROM res;
```



```
ALTER TABLE "EVENEMENTS" DROP CONSTRAINT "EVENEMENTS_pk"  
CASCADE;
```

```
INSERT INTO "EVENEMENTS" ("idEvènement", "évènement",  
"debut", "fin", "mesure") SELECT DISTINCT code, type_even, debut, fin, mesure  
FROM info;
```

```
ALTER TABLE "SITES" DROP CONSTRAINT "SITES_pk" CASCADE;
```

```
INSERT INTO "COMMUNES" (code_commune, nom, "idSite_SITES") SELECT  
DISTINCT code_INSEE, commune, code FROM sit;
```

```
INSERT INTO "DEPARTEMENTS"  
("code_departement", "nom_departement", "code_region_REGIONS") SELECT  
DISTINCT code, nom, code_region FROM depart_fr ; INSERT 0 101
```

Description commentée des différentes étapes de votre script de peuplement

Etape 1 : Création de quatres tables temporaire pour y insérer le contenu des quatres fichier

Etape 2 : Insertion du contenu de chaque fichier dans leur tables

Etape 3 : Projection dans la table REGIONS

Etape 4 : Projection dans la table SITES seulement des attributs : idSite, nom_site, dateDeclaration, typeEau, longitude et latitude. Il reste idAnalyse_ANALYSES et idEvènement_EVENTEMENT auquel la projection n'a pas été un succès.

Etape 5 : Suppression de la clé primaire dans la table ANALYSES

Etape 6 : Projection de la table ANALYSES

Etape 7 : Suppression de la clé primaire dans la table EVENEMENTS

Etape 8 : Projection de la table EVENEMENTS

Etape 9 : Suppression de la clé primaire dans la tabe SITES

Etape 10 : Projection dans la table COMMUNES

Etape 11 : Projection dans la table DEPARTEMENTS seulement des attributs : code_departement, nom_departement et code_region_REGIONS. Il manque l'attribut code_commune_COMMUNES.